- The three basic SQA management tools offered here are:
  a) Project progress control
  b) Software quality metrics
  c) Software quality costs

Management Component of Software Quality

Chapter-20- Project Progress Control

The components of management's control of project progress

- There are four main components of project progress control. Management is expected to intervene and contribute to arriving at solutions in extreme cases.

  (a) Control of risk management

  (b) Project schedule control

  (c) Project resource control

  (d) Project budget control

- **Control of risk management** activities refers to actions taken with respect to software risk items identified in the contract review and project plan documents as well as to risk items identified later, during the project's progress.
  - In practice, the software development team attempts to reduce risk by applying systematic risk management activities.
  - Management controls these efforts through review of periodic reports and evaluation of progress information.
  - This component of progress control directly contributes to achievement of the project's functional and technical objectives.
- **Project schedule control** deals with compliance with the project's approved and contractual timetables.
  - Follow-up is based on milestones in addition to periodic reports, which together enable identification of delays in completion of planned activities.
  - Special emphasis is given to customer-demanded milestones, as noted in the contract.
  - Management tends to focus control on those critical delays that threaten to substantially interfere with project completion dates.

- **Project resource control** focuses on professional human resources; it also deals with software development and testing facilities, typically required by real-time software systems and firmware.
  - Management exercises control on the basis of periodic reports of resources used, which should be viewed in terms of actual project progress.
- **Project budget control** is based on comparison of actual with scheduled costs. The main budget items to be controlled are:
  - Human resources
  - Development and testing facilities
  - Purchase of COTS software
  - Purchase of hardware
  - Payments to subcontractors

Management Component of Software Quality

Chapter 21 - Software quality metrics

Main objectives of software quality metrics

- To facilitate management control as well as planning and execution of the appropriate managerial interventions.
- To identify situations that require or enable development or maintenance process improvement in the form of preventive or corrective actions introduced throughout the organization.

- The metrics are used for comparison of performance data with indicators, quantitative values such as:
  - Defined software quality standards
  - Quality targets set for organizations or individuals
  - Previous year's quality achievements
  - Previous project's quality achievements
  - Average quality levels achieved by other teams applying the same development tools in similar development environments
  - Average quality achievements of the organization
  - Industry practices for meeting quality requirements.

Frame 21.2 Software quality metrics – requirements

General requirements	Explanation
Relevant	Related to an attribute of substantial importance
Valid	Measures the required attribute
Reliable	Produces similar results when applied under similar conditions
Comprehensive	Applicable to a large variety of implementations and situations
Mutually exclusive	Does not measure attributes measured by other metrics

Operative requirements	Explanation
Easy and simple	The implementation of the metrics data collection is simple and is performed with minimal resources
Does not require independent data collection	Metrics data collection is integrated with other project data collection systems: employee attendance, wages, cost accounting, etc. In addition to its efficiency aspects, this requirement contributes to coordination of all information systems serving the organization
Immune to biased interventions by interested parties	In order to escape the expected results of the analysis of the metrics, it is expected that interested persons will try to change the data and, by doing so, improve their record. Such actions obviously bias the relevant metrics. Immunity (total or at least partial) is achieved mainly by choice of metrics and adequate procedures

Classification of software quality metrics

- Software quality metrics can fall into a number of categories. Here we use a two-level system. The first classification category distinguishes between life cycle and other phases of the software system:
  - Process metrics, related to the software development process
  - Product metrics, related to software maintenance

  The second classification category refers to the subjects of the measurements:

  ■ Quality

  ■ Timetable

  ■ Effectiveness (of error removal and maintenance services)

  ■ Productivity.

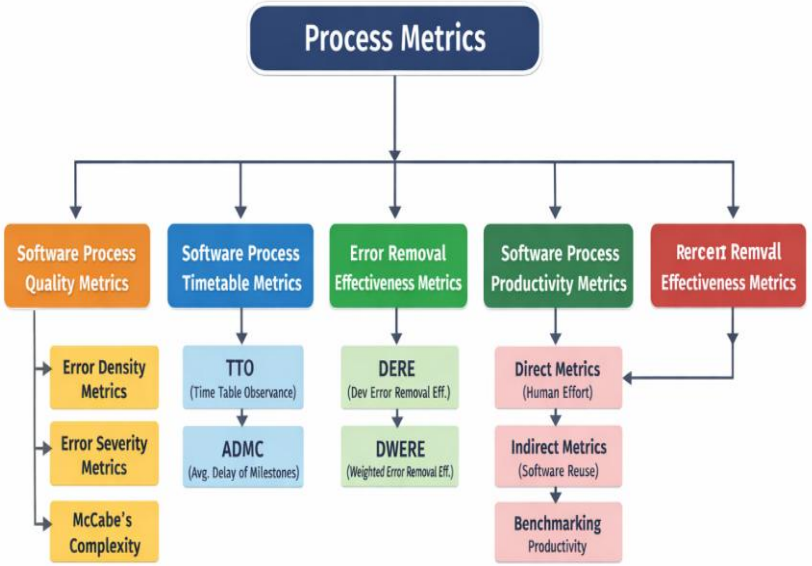
  These items are dealt with in the respective sections. A sizeable number of software quality metrics involve one of the two following measures for system size: KLOC & Function points

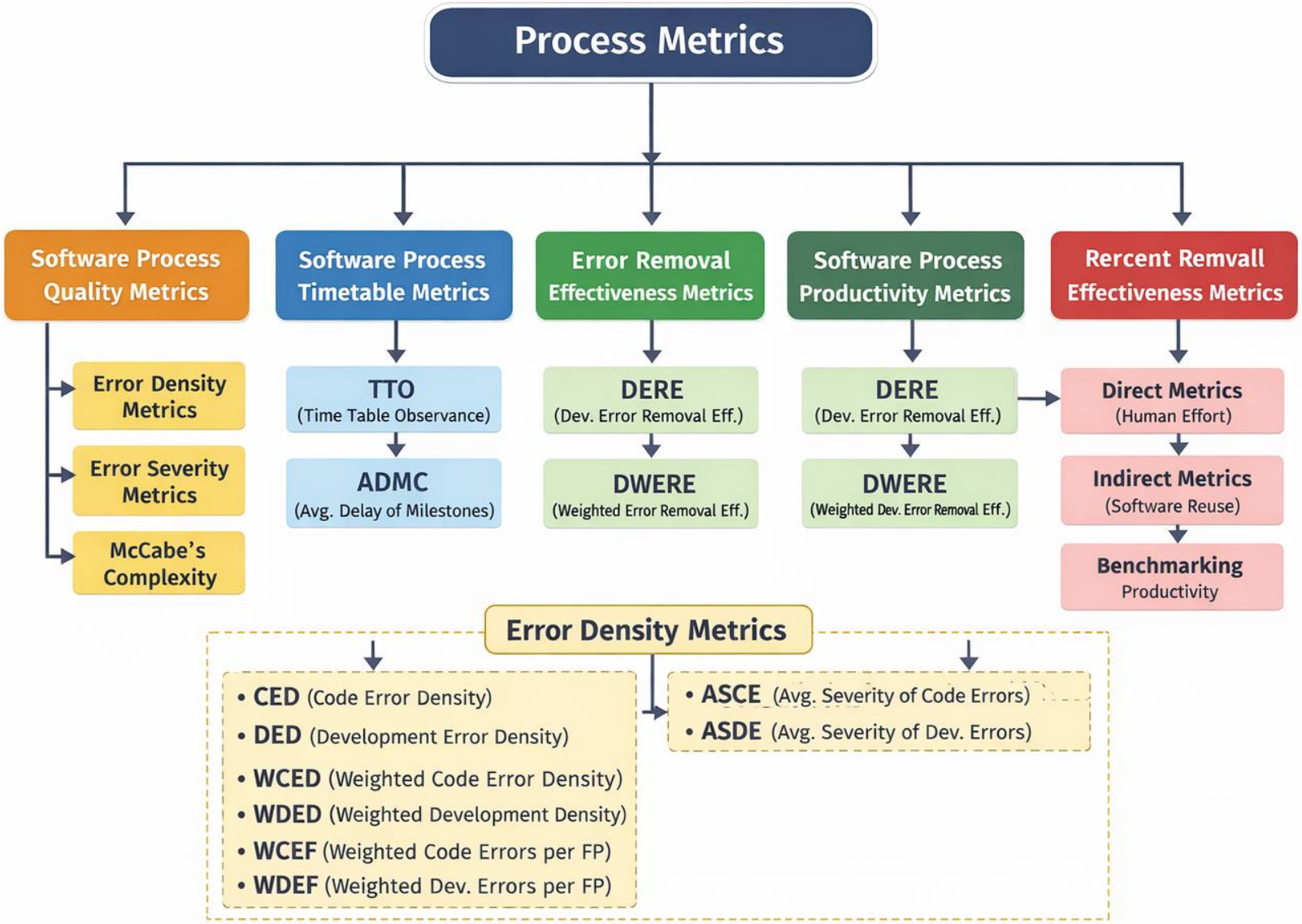
■ KLOC– this classic metric measures the size of software by thousands of code lines. Application of metrics that include KLOC is limited to software systems developed using the same programming language or development tool.

■ Function points– a measure of the development resources (human resources) required to develop a program, based on the functionality specified for the software system

Process metrics

- Software development process metrics can fall into one of the following categories:
  1. Software process quality metrics
  2. Software process timetable metrics
  3. Error removal effectiveness metrics
  4. Software process productivity metrics.




Process Metrics
Software Process Quality Metrics
Software Process Timetable Metrics
Error Removal Effectiveness Metrics
Software Process Productivity Metrics
Rercent Remvall Effectiveness Metrics
Error Density Metrics
Error Severity Metrics
McCabe's Complexity
TTO
(Time Table Observance)
ADMC
(Avg. Delay of Milestones)
DERE
(Dev. Error Removal Eff.)
DWERE
(Weighted Error Removal Eff.)
DERE
(Dev. Error Removal Eff.)
DWERE
(Weighted Dev. Error Removal Eff.)
Direct Metrics
(Human Effort)
Indirect Metrics
(Software Reuse)
Benchmarking
Productivity
Error Density Metrics
• CED (Code Error Density)
• DED (Development Error Density)
• WCED (Weighted Code Error Density)
• WDED (Weighted Development Density)
• WCEF (Weighted Code Errors per FP)
• WDEF (Weighted Dev. Errors per FP)
• ASCE (Avg. Severity of Code Errors)
• ASDE (Avg. Severity of Dev. Errors)

Software process quality metrics

- Software process quality metrics may be classified into two classes:
    1. Error density metrics
    2. Error severity metrics.
- Calculation of error density metrics involves two measures:
  (1) software volume
  (2) errors counted.

Product metrics

- Product metrics refer to the software system's operational phase.
- During the software's operational phase. Customer services are of two main types:
  - Help desk services (HD)
  - Corrective maintenance services

Help desk services (HD)

- Help desk services (HD)– software support by instructing customers regarding the method of application of the software and solution of customer implementation problems.
- Demand for these services depends to a great extent on the quality of the user interface (its “user friendliness”) as well as the quality of the user manual and integrated help menus.

Corrective maintenance services

- Corrective maintenance services– correction of software failures identified by customers/users or detected by the customer service team prior to their discovery by customers.
- The number of software failures and their density are directly related to software development quality.
- For completeness of information and better control of failure correction, it is recommended that all software failures detected by the customer service team be recorded as corrective maintenance calls.

Product metrics classification

- HD quality metrics
- HD productivity and effectiveness metrics
- Corrective maintenance quality metrics
- Corrective maintenance productivity and effectiveness metrics.

Software maintenance activities include:

- Corrective maintenance – correction of software failures detected during regular operation of the software.
- Adaptive maintenance – adaptation of existing software to new customers or new requirements.
- Functional improvement maintenance – addition of new functions to the existing software, improvement of reliability, etc.

HD quality metrics

- HD calls density metrics – the extent of customer requests for HD services as measured by the number of calls.
- Metrics of the severity of the HD issues raised.
- HD success metrics – the level of success in responding to these calls.

A success is achieved by completing the required service within the time determined in the service contract.

Table 21.6: HD calls density metrics

Code	Name	Calculation formula
HDD	HD calls Density	$HDD = \frac{NHYC}{KLMC}$
WHDD	Weighted HD calls Density	$WHDD = \frac{WHYC}{KLMC}$
WHDF	Weighted HD calls per Function point	$WHDF = \frac{WHYC}{NMFP}$

Key:

- NHYC = number of HD calls during a year of service.
- KLMC = thousands of lines of maintained software code.
- WHYC = weighted HD calls received during one year of service.
- NMFP = number of function points to be maintained.

Corrective maintenance quality metrics

■ Software system failures density metrics– deal with the extent of demand for corrective maintenance, based on the records of failures identified during regular operation of the software system.

■ Software system failures severity metrics– deal with the severity of soft ware system failures attended to by the corrective maintenance team.

Table 21.8: Software system failures density metrics

Code	Name	Calculation formula
SSFD	Software System Failure Density	$SSFD = \frac{NYF}{KLMC}$
WSSFD	Weighted Software System Failure Density	$WSSFD = \frac{WYF}{KLMC}$
WSSFF	Weighted Software System Failures per Function point	$WSSFF = \frac{WYF}{NMFP}$

Key:

- NYF = number of software failures detected during a year of maintenance service.
- WYF = weighted number of yearly software failures detected during a year of maintenance service.
- KLMC = thousands of lines of maintained software code.
- NMFP = number of function points designated for the maintained software.

Software system availability metrics

- User metrics distinguish between:

■ Full availability – where all software system functions perform properly

■ Vital availability – where no vital functions fail (but non-vital functions may fail)

■ Total unavailability – where all software system functions fail.

The function point method

- An important attribute of the function point method is its capacity to pro vide pre-project estimates of project size, stated in terms of required development resources.
- KLOC cannot be used for early size estimation because it is measured **only after coding is completed**, which is a late project stage.
- The **Function Point Method** overcomes this limitation by measuring software size based on **functionality defined in customer requirements**.

Programming language/development tool	Average LOC
C	128
C++	64
Visual Basic	32
Power Builder	16
SQL	12

- The function point method for estimating project size is conducted as follows:
- Stage 1: Compute crude function points (CFP). The number of software system functional components are first identified, followed by evaluation of each component as “simple”, “average” or “complex”
- Stage 2: Compute the relative complexity adjustment factor (RCAF) for the project. The RCAF varies between 0 and 70.
- Stage 3: Compute the number of function points (FP):

$$FP = CFP \times (0.65 + 0.01 \times RCAF)$$

Table 21A.1: Crude Function Points (CFP) – calculation form

Software System Component	Complexity level									Total CFP
	Simple			Average			Complex			
	Count	Weight factor	Points	Count	Weight factor	Points	Count	Weight factor	Points	
	A	B	C= A×B	D	E	F= D×E	G	H	I= G×H	J=C+F+I
User inputs		3			4			6		
User outputs		4			5			7		
User online queries		3			4			6		
Logical files		7			10			15		
External interfaces		5			7			10		
Total CFP										

- Stage 1: Calculation of crude function points
  - Number of user inputs – distinct input applications, not including inputs for online queries.
  - Number of user outputs – distinct output applications such as batch processed reports, lists, customer invoices and error messages (not includ ing online queries).
  - Number of user online queries – distinct online applications, where out put may be in the form of a printout or screen display.
  - Number of logical files – files that deal with a distinct type of data and may be grouped in a database.
  - Number of external interfaces – computer–readable output or inputs transmitted through data communication, on CD, diskette, etc.

- Stage 2: Calculating the relative complexity adjustment factor (RCAF) 445 The relative complexity adjustment factor (RCAF) summarizes the complexity characteristics of the software system by assigning grades (0 to 5) to the 14 subjects that substantially affect the required development efforts. The list of subjects is presented in the RCAF calculation form; see Table 21A.2.

Table 21A.2: Relative Complexity Adjustment Factor (RCAF) – calculation form

No.	Subject	Grade
1	Requirement for reliable backup and recovery	0 1 2 3 4 5
2	Requirement for data communication	0 1 2 3 4 5
3	Extent of distributed processing	0 1 2 3 4 5
4	Performance requirements	0 1 2 3 4 5
5	Expected operational environment	0 1 2 3 4 5
6	Extent of online data entries	0 1 2 3 4 5
7	Extent of multi-screen or multi-operation online data input	0 1 2 3 4 5
8	Extent of online updating of master files	0 1 2 3 4 5
9	Extent of complex inputs, outputs, online queries and files	0 1 2 3 4 5
10	Extent of complex data processing	0 1 2 3 4 5
11	Extent that currently developed code can be designed for reuse	0 1 2 3 4 5
12	Extent of conversion and installation included in the design	0 1 2 3 4 5
13	Extent of multiple installations in an organization and variety of customer organizations	0 1 2 3 4 5
14	Extent of change and focus on ease of use	0 1 2 3 4 5
	Total = RCAF	

- Stage 3: Computing the number of function points (FP) The function point value for a given software system is computed according to the results of stages 1 and 2, by applying the following formula:

$$FP = CFP \times (0.65 + 0.01 \times RCAF)$$

- Number of user inputs – 2
- Number of user outputs – 3
- Number of user online queries – 3
- Number of logical files – 2
- Number of external interfaces – 2.

Table 21A.3: Attend-Master Crude Function Points (CFP) – calculation form

Software System Component	Compexity level									Total CFP
	Simple			Average			Complex			
	Count	Weight factor	Points	Count	Weight factor	Points	Count	Weight factor	Points	
	A	B	C= A×B	D	E	F= D×E	G	H	I= G×H	J=C+F+I
User inputs	1	3	3	---	4	---	1	6	6	**9**
User outputs	---	4	---	2	5	10	1	7	7	**17**
User online queries	1	3	3	1	4	4	1	6	6	**13**
Logical files	1	7	7	---	10	---	1	15	15	**22**
External interfaces	---	5	---	---	7	---	2	10	20	**20**
Total CFP										**81**

Function point advantages and disadvantages

Main Advantages

- Can be estimated at the pre-project stage, helping in early planning.
- Based on requirement specifications, not programming language.
- Therefore, it is language-independent and relatively reliable.

Main Disadvantages

- Results may vary depending on the counting manual used.
- Requires detailed specifications, which may not always be available early.
- Needs an experienced team and significant resources.
- Involves subjective evaluations.
- Mainly suitable for data processing systems; not universally applicable.

Limitations of software metrics

- General Limitations
  - Budget constraints – Lack of funds and manpower to implement metrics properly.
  - Human resistance – Employees may oppose performance measurement.
  - Data reliability issues – Incomplete, biased, or inconsistent reporting.
- Software-Specific Limitations
  - Most metrics have low validity (do not truly measure quality/productivity).
  - Metrics show limited comprehensiveness (do not cover all quality aspects).
  - Results are often distorted by external factors.

Management Component of Software Quality

Chapter 22 – Cost of Software Quality

Cost of software quality metrics– objectives

- Application of cost of software quality metrics enables management to achieve economic control over SQA activities and outcomes. The specific objectives are:
  - Control organization-initiated costs to prevent and detect software errors
  - Evaluation of the economic damages of software failures as a basis for revising the SQA budget
  - Evaluation of plans to increase or decrease SQA activities or to invest in a new or updated SQA infrastructure on the basis of past economic performance

Compare the classic software quality costs model with the extended model.

- A general concept that classifies manufacturing quality costs into two classes:
  - costs of control (prevention costs and appraisal costs)
  - costs of failure of control (internal failure costs and external failure costs)

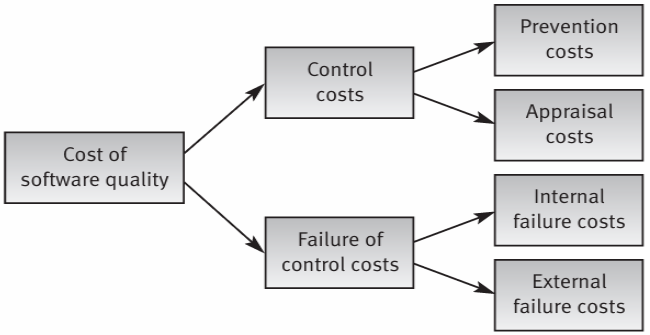


Figure 22.1: The classic model of cost of software quality

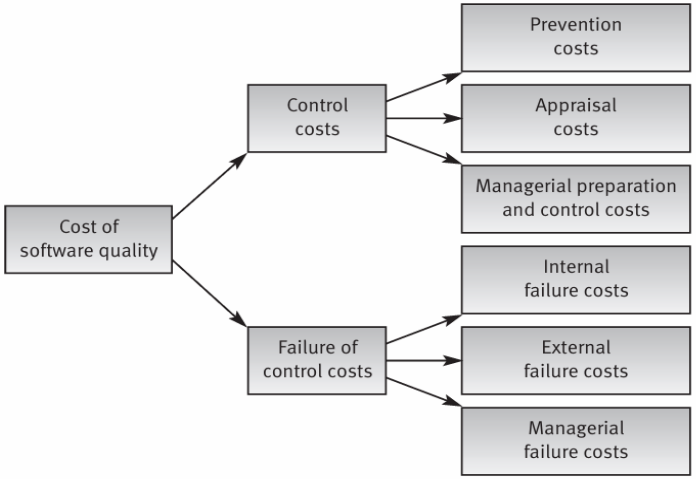


Figure 22.2: The extended cost of software quality model

- **costs of control (prevention costs and appraisal costs)** – costs controlled by the organization and expended to prevent and detect failures so as to reduce total failures to an acceptable level.
- **costs of failure of control (internal failure costs and external failure costs) -** costs of failures, regarded as consequences, caused by failure to prevent and detect software errors.
- The extended model expands the scope of the classic model by introducing factors related to management's contribution to project success and failure.
- The subclasses added are managerial preparation and control costs (a third subclass to the costs of control class), and managerial failure costs (a third subclass to the costs of failure of control class).

Table 22.3: Typical causes for delays and associated costs

Cause for deviation from schedule	Class of quality costs
1. Changes introduced in the project's specifications during development	No internal failure costs; customer responsibility for failure costs
2. Customer-delayed installation of communication and other hardware, and/or delays in staff recruitment and training	No internal failure costs, customer responsibility for failure costs
3. Poor performance by development team, requiring extensive rework and corrections of software	External failure costs
4. Project proposal based on unrealistic schedules and budgets	Managerial failure costs
5. Late or inadequate recruitment of staff or reliance on company professionals whose release from other projects does not meet project needs	Managerial failure costs